

2 Flirds 체크포인트 02 — 최종 실험 세팅

02 · 최종 실험 세팅 (세부)

결정된 값 + 그 결정의 이유/흔적. 코드 우선(plan 서술과 갈리면 코드가 정답). 태그 [CODE](#) / [DOC](#) / [실측](#)

2.1 모델 · regime · seed

축	값	근거
모델	1B = <code>meta-llama/Llama-3.2-1B-Instruct</code> (16L) · 3B = Llama-3.2-3B-Instruct (28L) · 7B = Llama-2-7B	[DOC <code>flirds.md:69</code>]; [CODE <code>phase2_matrix.py:80</code>]
regime	silos = cross-silo N=5 (full participation, 도메인당 1 client) · device100 = cross-device N=100 (Dirichlet- α partial, per-client 도메인 혼합, K=10/round)	[CODE <code>phase2_matrix.py:103,109</code> ; <code>llm.py:167</code>]
seed	3개 <code>[0,1,2]</code> (protocol 기본 <code>[42,123,2024]</code> 과 다름 — 코드가 <code>seeds=[0,1,2]</code>)	[CODE <code>phase1_clean_run.py:59</code> ; <code>phase2_matrix.py:375</code>]
α -sweep (device100)	<code>{0, 0.01, 0.1, 0.5, 5.0}</code> ; in-run oracle은 $\alpha=0.5$ 앵커 1점만	[DOC plan §3.9]; [CODE <code>phase2_matrix.py:86</code>] ALPHA env]
scale-up scale 메모리	1B batch16/val_chunk10 · 3B batch8/val_chunk5 · 7B batch4/val_chunk2, 전부 fp32 (7B도 bf16 안 씀 — bf16은 연기된 retrain oracle용)	[CODE <code>phase2_matrix.py:97-100</code>]

est-vs-oracle 비교 매트릭스 [DOC](#): LLM 1B retrain & in-run oracle N=5 / N=10 연기. LLM 3B N=5만. LLM 7B in-run oracle N=5, retrain oracle X. estimator METHOD(noisy-AUROC-selection, oracle 불필요)는 N=10서도 가능. CNN oracle 검증은 plan §3.11 **Track C1** — MNIST+LeNet5/CIFAR-10+FedSVCNN, **N=10 full**, 듀얼 oracle((a) 2^{10} retrain val-loss + (b) exact in-run), 구현 완료 [a](#) (실측 없음). LLM 추가분은 **Track D** 표준세팅(전부 API-free, 설계만 [c](#); 7B=Llama-2-7b-hf).

2.2 데이터 레이어 (5-domain cross-silo)

`data/llm.py` — `build(n_clients, per_domain_train, per_domain_val, per_domain_test, seed, noisy=, backdoor=)` → `(clients, val_records, test_records)`. `N∈{5,10}` assert (`llm.py:167`). N=5 → 도메인당 1 client; N=10 → 도메인당 2 client(disjoint 절반).

도메인	HF dataset	val 출처	근거
medical	<code>medalpaca/medical_meadow_medical_flashcards</code>	train-carve	[CODE <code>llm.py:69-80</code>]

도메인	HF dataset	val 출처	근거
legal	<code>ibunesu/qa_legal_dataset_train</code>	train-carve	"
finance	<code>LLukas22/fiqa</code>	native <code>test</code>	"
math	<code>deepmind/aqua_rat</code> (config <code>raw</code> , rationale)	native <code>validation</code>	"
general	<code>databricks/databricks-dolly-15k</code>	train-carve	"

- 크기: per-domain train=**12,000** / val=**200**(총 1,000) / test=**2,000**, 상호 disjoint [CODE `phase1_clean_run.py:58-59`]. val=1,000 ≠ 1,024(=2¹⁰ coalition) 의도적 분리 [DOC plan §3.4].
- 5 도메인 모두 free-form instruction→response** (cross-domain 포맷 균일성 = 노벨티 hook; 이질 포맷이면 shared-val-loss Shapley가 불공정) [DOC `threads/dataset-format-uniformity`].
- equalized train = size CONTROL 변수 (B1) DOC**.
- per-domain normalization 옵션** (`backends/llm.py:80-88`): token-norm `n_c/Ntot` (기본) vs domain-macro `n_c/(D·n_d)` (ablation arm). domain-norm은 φ 순서를 눈에 띄게 바꿈 CODE.

`device100: build_crossdevice(n_clients, alpha, per_client, per_domain_pool, ...)` via `fl.partition.client_dirichlet_partition` (per-CLIENT Dir(α) 도메인-혼합 = **Option B**; 고정크기-전부 non-empty- $\alpha=0$ =도메인 disjoint) [CODE `partition.py:49-82`]. `per_client = 300` (40→300 변경; [04](#) 참조) [CODE `phase2_matrix.py:109`].

2.3 LoRA · 학습 하이퍼 · fp32 이유

[CODE `phase1_clean_run.py:124-125`, `:44`, `:58`; `llm_server.py:46,52-53`]

```
LoraConfig(r=16, lora_alpha=32,
            target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","down_proj"],
            lora_dropout=0.0, task_type="CAUSAL_LM")
# 학습: SGD momentum=0 (forced), constant lr, completion_only_loss=True; bf16=False, fp16=False
```

- phase1 FULL**: `rounds=50, max_steps=10, lr=2e-5, batch=16, maxlen=768, val_maxlen=384, val_chunk=10` [CODE `phase1_clean_run.py:58`].
- phase2 matrix 기본(smoke)**: silo5 `lr=1e-3, rounds=10, max_steps=10, batch=16, val=20`; device100 `rounds=30, max_steps=5, val=10` — coarse smoke값 [CODE `phase2_matrix.py:103,109`].
- SGD momentum=0 강제** [CODE `llm_server.py:52-53`; `codes/CLAUDE.md §5`]: IRDS/Ripple per-step 가정; momentum 켜면 2차 Taylor 열화.
- fp32 master, 왜 bf16 불가**: utility=coalition val-loss 차 ~0.005-0.02 < bf16 정밀도 ~0.009 → bf16서 Spearman 무의미. fp32서 +1.000 [CODE `oracle/exact_sv_llm.py:62-63`; B raw `2026-06-07-phase2-task6-a-retrain-oracle.md`]. eval forward도 fp32(HVP 수치안정).
- attn_implementation="eager"** [CODE `phase1_clean_run.py:123`]: forward-mode AD HVP가 SDPA/flash서 NotImplementedError.

△ **plan-vs-code divergence**: plan §3.3은 phase1 lr=2e-5(fine-tune scale), matrix는 비교실험용 lr=1e-3/2e-3(빠른 수렴). real grid는 06-10 시작 tier1(silo5 4-threat) 완료 — noisy/FR=lr1e-3/batch16, poison=working-backdoor lr2e-3/batch8(전부 R=10)로 확정 실행([07 §7.0](#) 참조). 22/25셀 metrics.json이 `runs/phase2_matrix/rundirs` 에 영속화됨(남은 3셀=dev_a0.5 anchor {noisy,frand,frzero}). [04](#) 참조.

2.4 위협 4종 정의 — 어떤 논문 근거로 왜 그렇게 정의했나

`data/corruptors.py` + `fl/llm_server.py` . 상세 PDF 대조 → 03.

noisy (data-quality) — `answer_swap` [CODE `corruptors.py:28-39`]

- 무엇: client 내부에서 completion 컬럼을 permute → 각 prompt가 다른 행의 answer와 짝. prompt 불변.
- 왜 이렇게: "정직하지만 나쁜 데이터" = data-quality 위협 (악의 아님). FedDQC의 IRA가 정확히 이걸 잡도록 설계 — 매칭 detector = FedDQC [DOC `threads/data-quality-vs-data-value`].

free-rider — `free_rider(ref, mode)` [CODE `corruptors.py:70-93`]

- zero: $\Delta w=0 \rightarrow \varphi$ 정확히 0 (자명검출). random: $\Delta w \sim U(-scale, scale)$, benign-std 매칭(`scale=benign_std*√3`, `phase2_matrix.py:219-220`) → $\varphi \approx 0$.
- 왜 이렇게: Lin et al. 2019 (STD-DAGMM 원논문, 1911.12560) free-rider attack taxonomy의 easy 쪽 PDF . 어려운 delta/advanced-delta는 advanced free-rider로 연기 (이전 global model을 FL 루프에 threading 필요).

poison (malicious) — `backdoor()` + `scaled_attackers` [CODE `corruptors.py:47-63` + `llm_server.py:57-58`]

- 무엇: Xu 2023 instruction-trigger "tq" → target string, `poison_frac` 만큼 poison (clean-preservation knob) [PDF web-extract 2305.14710]. + Bagdasaryan 2020 plain-scaled model-replacement `delta×attack_scale` ($\gamma = \text{cohort size} = n/\eta$) [PDF web-extract 1807.00459].
- 왜 이렇게: backdoor는 새 위협(원 plan에 없던). Xu=instruction-tuning backdoor 표준, Bagdasaryan=FL 전파 메커니즘. DBA(Xie2020, 다중공모)는 제외. 우리는 stealthy constrain-and-scale 안 함 — plain-scaled만(attacker $\|\Delta\| = 40 \times \text{benign} \rightarrow \text{norm-bound가 backdoor 죽음} \rightarrow \text{stealthy arm 불가}$).
- working-backdoor config 필요: 전파엔 `LR=2e-3 BATCH=8 EPOCHS=5 POISON_FRAC=0.8` [CODE `phase2_matrix.py:40-44`]; matrix 기본 `lr1e-3/batch16`은 ASR=0(batch16이 attacker install step 절반).

왜 검출을 하나 (우리가 라벨을 주입하는데): method는 라벨에 blind; 라벨은 평가 KEY(AUROC)지 method 입력이 아님 → 순환 아님. 두 목적: (1) value SEMANTICS 검증(corrupt→low value; oracle은 "Shapley 계산 일치"만 증명) (2) 전용 detector 대비 경쟁 bar [DOC `MEMORY.md` detector 섹션].

2.5 in-run oracle 비용 공식

$U_{(b)}(S) = \sum_r [\ell(w^r + \sum_{k \in S \cap P_r} p_k^r \Delta w_k) - \ell(w^r)]$, exact 2^N enumeration [CODE `in_run_sv.py:3-6, :71-104`].

- 비용 = $2^N \cdot R \cdot \text{val} \cdot \text{seq}$, FLOP-bound [DOC `flirds-protocol.md:90` ; CODE `phase1_clean_run.py:52-54`]. $N_5 \leftrightarrow N_{10} = 32\times$.
- 실측 cross-device: oracle 771ms/fwd (fp32-B200, no-tensor-core) → R=200이면 ~44h/1-GPU → ~11h/4-GPU → α 1-2점만 [ raw 2026-06-08-...redesign.md].
- cross-device는 `in_run_shapley_perround` = round별 $2^{|P_r|}$ 분해 (2^N 과 동일, $\Delta\varphi \approx 3e-16$) → N=100서 2^{100} 대신 200×1024 [CODE `in_run_sv.py:126-165`].

2.6 검출 정보를 성능 향상에 쓰는 법 (selection/filtering)

[CODE `phase1_clean_run.py:88-90, :157-158`]

```
def select_topk(phi, k):    # 가장 낮은 ϕ k개 = 가장 가치있는 client (val-loss 최대 감소)
    return sorted(range(len(phi)), key=lambda i: phi[i])[:k]
arms = {"full": all_idx, "flirds_topk": keep, "random_k": rand}    # K=3 (N=5서 2개=noisy+FR 드롭)
```

성공기준: `flirds_topk val_loss ≤ random_k` and `ROUGE-L ≥ random_k` [CODE `read_runs.py:53`].

selection run 실측 결과

⑥ 1B N=5 cross-silo, 3-seed, lr 1e-3 & 3e-3

직접 `codes/runs/full_lr{1e-3,3e-3}/.../metrics.json` 6개 파일 대조:

	lr=1e-3 (seed 0/1/2)	lr=3e-3 (seed 0/1/2)
flirds_keep (set)	{2,3,4} 모두 동일	{2,3,4} 모두 동일
client1 (free-rider) ϕ	0.0 exact 매 seed	0.0 exact 매 seed
AUROC noisy / FR	0.75 / 1.0 (3 seed 동일)	1.0 / 0.75 (3 seed 동일)
flirds_topk vs full	항상 이김 (예 s0 2.40398 < 2.41421)	항상 이김 (단 margin 작음, s0 2.39724 < 2.39759)
flirds_topk vs random_k	s0 tie(random도 {2,3,4}), s1-s2 이김	s0 tie, s1-s2 이김

확인된 것: Flirds가 noisy(c0)+free-rider(c1)를 정확히 드롭하고 clean {2,3,4}만 keep — 양 lr-전 seed 일관. free-rider $\phi=0$ exact.

정직한 nuance 2개 (직접 metrics.json 대조로 발견):

- seed0은 random이 우연히 같은 clean set {2,3,4}를 골라 tie (val_loss curve 동일). "beats random" 근거는 seed1/2 (random이 corrupted 포함 시: s1 random={1,2,3} FR포함→짐, s2 random={0,1,2} noisy+FR포함→짐). → "beats random"은 cross-seed 평균 주장이지 매-seed 아님.
- AUROC가 lr로 뒤집힘 + run간 분산: 이 run에서는 lr1e-3 noisy 0.75/FR 1.0 ↔ lr3e-3 noisy 1.0/FR 0.75. noisy client(c0) ϕ 부호가 lr 의존(lr1e-3서 -0.0084, lr3e-3서 +0.0096). selection(flirds_keep)은 양 lr 동일하므로 결론 불변. 단, 06-10 영속화 real-grid run(`runs/phase2_matrix/rundirs/silo5_noisy`, 동일 lr=1e-3)에서는 Flirds noisy AUROC 1.000 ± 0.000 per-seed [1.0, 1.0, 1.0](FR random/zero도 1.000) — run간 분산이 큼(양쪽 다 실측, 어느 run도 무효 아님). 'lr1e-3=0.75'를 단일 run으로 단정하지 말 것.

신호는 작지만 일관 — "random은 어려운 bar(FedDQC/DsDm)" caveat는 (corrupted를 random이 포함하는 seed서) 통과.